

18장. IO기반 입출력 및 네트워킹

이것이 자바다

Contents

- 1절. IO 패키지 소개
- 2절. 입력 스트림과 출력 스트림
- 3절. 콘솔(Console) 입출력
- 4절. 파일(File) 입출력
- 5절. 보조 스트림
- 6절. 네트워크 기초
- 7절. TCP 네트워킹
- 8절. UDP 네트워킹



1절. IO 패키지 소개

❖ java.io 패키지

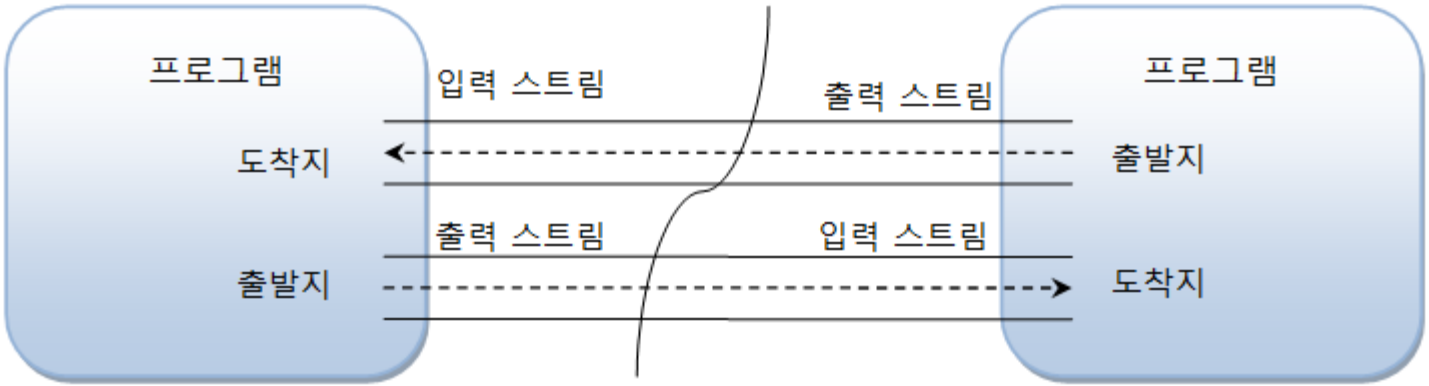
- 자바의 기본적인 데이터 입출력(IO: Input/Output) API 제공

java.io 패키지의 주요 클래스	설명
File	파일 시스템의 파일의 정보를 얻기위한 클래스
Console	콘솔로부터 문자를 입출력하기 위한 클래스
InputStream / OutputStream	바이트 단위 입출력을 위한 최상위 입출력 스트림 클래스
FileInputStream / FileOutputStream DataInputStream / DataOutputStream ObjectInputStream / ObjectOutputStream PrintStream BufferedInputStream / BufferedOutputStream	바이트 단위 입출력을 위한 하위 스트림 클래스
Reader / Writer	문자 단위 입출력을 위한 최상위 입출력 스트림 클래스
FileReader / FileWriter InutStreamReader / OutputStreamWriter PrintWriter BufferedReader / BufferedWriter	문자 단위 입출력을 위한 하위 스트림 클래스

2절. 입력 스트림과 출력 스트림

❖ 입력 스트림과 출력 스트림의 개념





2절. 입력 스트림과 출력 스트림

❖ 바이트 기반 스트림과 문자 기반 스트림

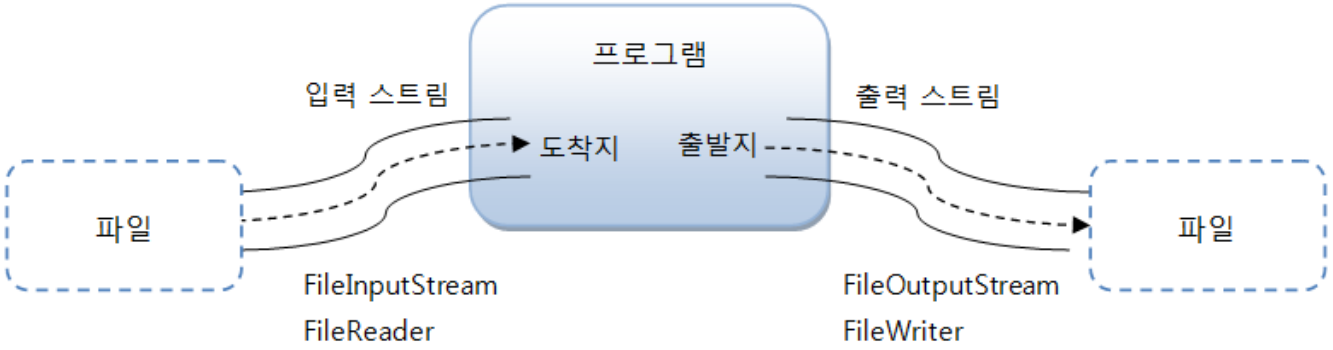
▪ 바이트 기반 스트림

- 그림, 멀티미디어, 문자 등 모든 종류의 데이터를 받고 보내는 것 가능

▪ 문자 기반 스트림

- 문자만 받고 보낼 수 있도록 특화

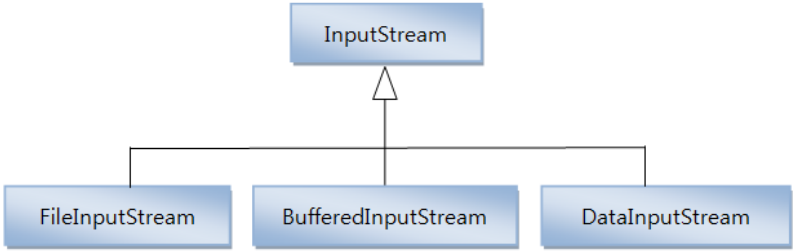
구분	바이트 기반 스트림		문자 기반 스트림	
	입력 스트림	출력 스트림	입력 스트림	출력 스트림
최상위 클래스	InputStream	OutputStream	Reader	Writer
하위 클래스 (예)	XXXputStream (FileInputStream)	XXXOutputStream (FileOutputStream)	XXXReader (FileReader)	XXXWriter (FileWriter)



2절. 입력 스트림과 출력 스트림

❖ InputStream

- **바이트 기반 입력 스트림의 최상위 클래스로 추상 클래스**



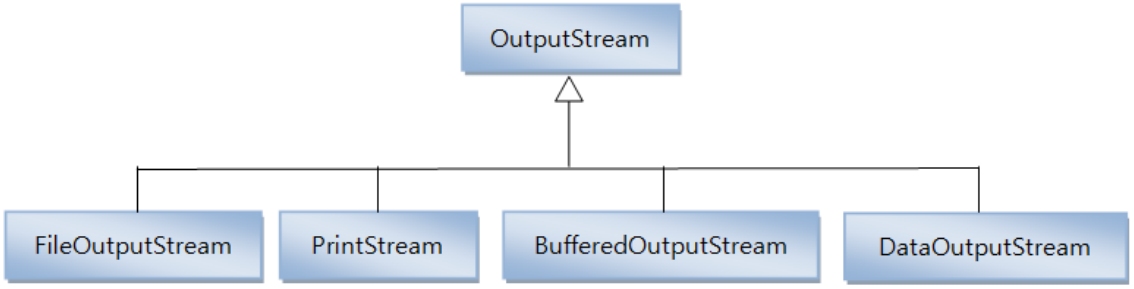
- **InputStream 클래스의 주요 메소드 (p.997~999)**

리턴타입	메소드	설명
int	read()	입력 스트림으로부터 1 바이트를 읽고 읽은 바이트를 리턴한다.
int	read(byte[] b)	입력 스트림으로부터 읽은 바이트들을 매개값으로 주어진 바이트 배열 b 에 저장하고 실제로 읽은 바이트 수를 리턴한다.
int	read(byte[] b, int off, int len)	입력 스트림으로부터 len 개의 바이트 만큼 읽고 매개값으로 주어진 바이트 배열 b[off] 부터 len 개까지 저장한다. 그리고 실제로 읽은 바이트 수인 len 개를 리턴한다. 만약 len 개를 모두 읽지 못하면 실제로 읽은 바이트 수를 리턴한다.
void	close()	사용한 시스템 자원을 반납하고 입력 스트림을 닫는다.

2절. 입력 스트림과 출력 스트림

❖ OutputStream

- 바이트 기반 출력 스트림의 최상위 클래스로 추상 클래스



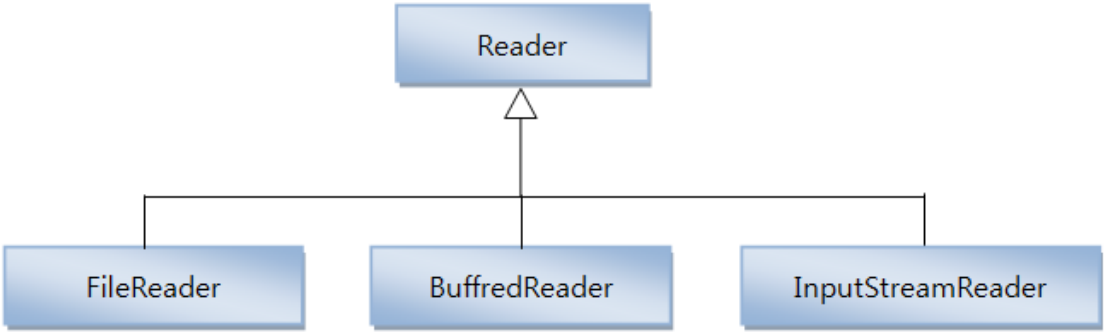
- OutputStream의 주요 메소드 (p.1000~1002)

리턴타입	메소드	설명
void	wrtie(int b)	출력 스트림으로 1 바이트를 보낸다.
void	write(byte[] b)	출력 스트림에 매개값으로 주어진 바이트 배열 b 의 모든 바이트를 보낸다.
void	write(byte[] b, int off, int len)	출력 스트림에 매개값으로 주어진 바이트 배열 b[off] 부터 len 개까지의 바이트를 보낸다.
void	flush()	버퍼에 잔류하는 모든 바이트를 출력한다.
void	close()	사용한 시스템 자원을 반납하고 출력 스트림을 닫는다.

2절. 입력 스트림과 출력 스트림

❖ Reader

- 문자 기반 입력 스트림의 최상위 클래스로 추상 클래스



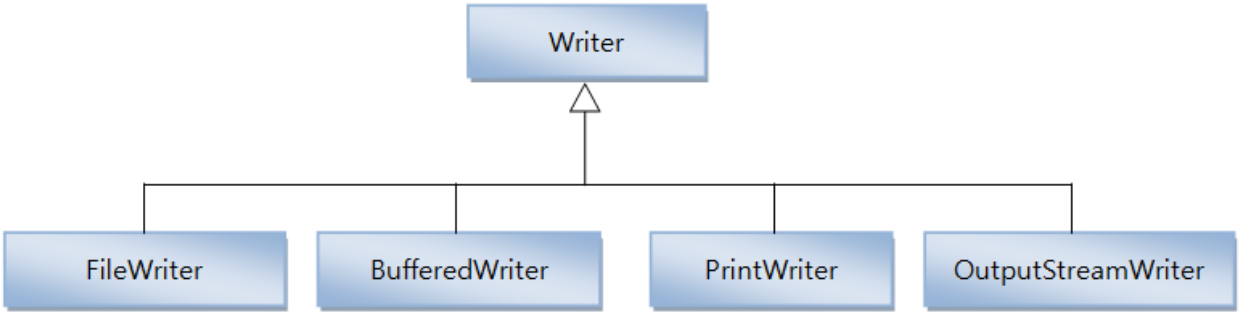
- Reader의 주요 메소드 (p.1002~1005)

메소드		설명
int	read()	입력 스트림으로부터 한개의 문자를 읽고 리턴한다.
int	read(char[] cbuf)	입력 스트림으로부터 읽은 문자들을 매개값으로 주어진 문자 배열 cbuf 에 저장하고 실제로 읽은 문자 수를 리턴한다.
int	read(char[] cbuf, int off, int len)	입력 스트림으로부터 len 개의 문자를 읽고 매개값으로 주어진 문자 배열 cbuf[off] 부터 len 개까지 저장한다. 그리고 실제로 읽은 문자 수인 len 개를 리턴한다.
void	close()	사용한 시스템 자원을 반납하고 입력 스트림을 닫는다.

2절. 입력 스트림과 출력 스트림

❖ Writer

- **문자 기반 출력 스트림의 최상위 클래스로 추상 클래스**



- **Writer의 주요 메소드 (p.1006~1009)**

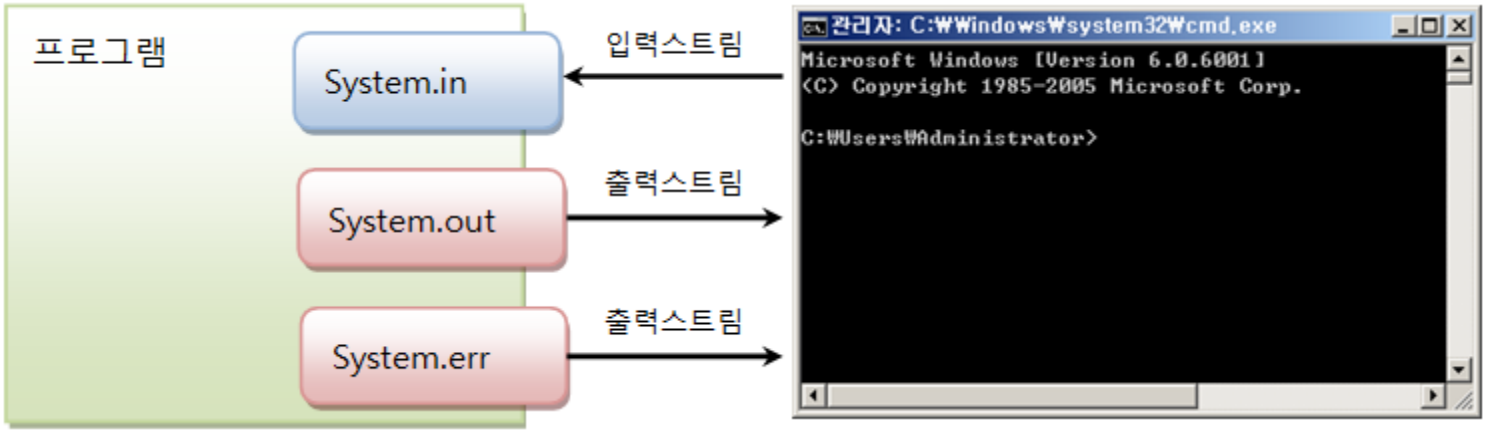
리턴타입	메소드	설명
void	wrtie(int c)	출력 스트림으로 매개값으로 주어진 한 문자를 보낸다.
void	write(char[] cbuf)	출력 스트림에 매개값으로 주어진 문자 배열 cbuf 의 모든 문자를 보낸다.
void	write(char[] cbuf, int off, int len)	출력 스트림에 매개값으로 주어진 문자 배열 cbuf[off] 부터 len 개까지의 문자를 보낸다.
void	write(String str)	출력 스트림에 매개값으로 주어진 문자열을 전부 보낸다.
void	write(String str, int off, int len)	출력 스트림에 매개값으로 주어진 문자열 off 순번부터 len 개까지의 문자를 보낸다.
void	flush()	버퍼에 잔류하는 모든 문자열을 출력한다.
void	close()	사용한 시스템 자원을 반납하고 출력 스트림을 닫는다.

3절. 콘솔 입출력

❖ 콘솔(Console)

- 시스템을 사용하기 위해 키보드로 입력을 받고 화면으로 출력하는 소프트웨어

- Unix, Linux: 터미널
- Windows 운영체제: 명령 프롬프트
- 이클립스: Console 뷰



3절. 콘솔 입출력

❖ System.in 필드

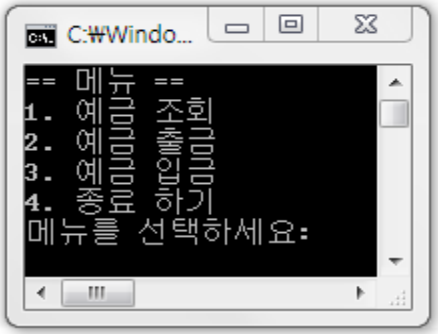
- InputStream 타입의 입력 스트림 – InputStream 변수 대입 가능
- 읽은 byte는 키보드의 아스키 코드(ascii code)
- 아스키 코드로부터 문자 변환

- 키보드로부터 입력된 한글 읽기 예제
  - read()메소드는 1바이트씩만 읽음 → 오류 발생
  - 전체 내용을 바이트 배열로 받아 String 객체 생성 후 읽기



SystemInExample.java 콘솔에서 입력한 번호 알아내기

```
1  import java.io.InputStream;
2
3  public class SystemInExample1 {
4      public static void main(String[] args) throws Exception {
5          System.out.println("== 메뉴 ==");
6          System.out.println("1. 예금 조회");
7          System.out.println("2. 예금 출금");
8          System.out.println("3. 예금 입금");
9          System.out.println("4. 종료 하기");
10         System.out.print("메뉴를 선택하세요: ");
11
12         InputStream is = System.in;
13         char inputChar = (char) is.read();
14         switch(inputChar) {
15             case '1':
16                 System.out.println("예금 조회를 선택하셨습니다.");
17                 break;
18             case '2':
19                 System.out.println("예금 출금를 선택하셨습니다.");
20                 break;
21             case '3':
22                 System.out.println("예금 입금를 선택하셨습니다.");
23                 break;
24             case '4':
25                 System.out.println("종료 하기를 선택하셨습니다.");
26                 break;
27         }
28     }
29 }
30
```



3절. 콘솔 입출력

- System.out 필드
  - PrintStream 타입의 출력 스트림
    - OutputStream으로 타입 변환 가능
  - 아스키 코드를 출력하면 콘솔에는 문자가 출력
  - 문자열을 출력하려면 바이트 배열을 얻어야



SystemOutExample.java 연속된 숫자, 영어, 한글 출력

```java
import java.io.OutputStream;

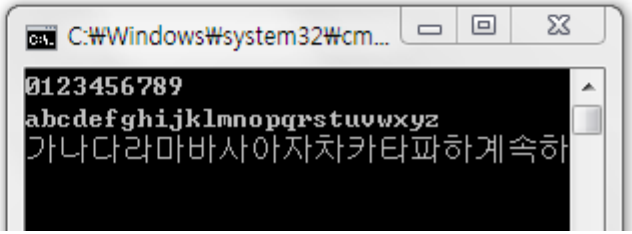
public class SystemOutExample {
    public static void main(String[] args) throws Exception {
        OutputStream os = System.out;

        for(byte b=48; b<58; b++) {
            os.write(b);
        }
        os.write(10);

        for(byte b=97; b<123; b++) {
            os.write(b);
        }
        os.write(10);

        String hangul = "가나다라마바사아자차카타파하";
        byte[] hangulBytes = hangul.getBytes();
        os.write(hangulBytes);

        os.flush();
    }
}
```



3절. 콘솔 입출력

❖ Console 클래스

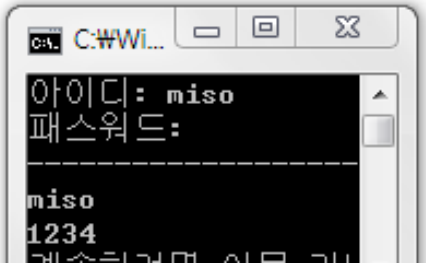
- 자바6부터 콘솔에서 입력된 문자열을 쉽게 읽을 수 있도록 제공
  - 이클립스에서 System.console()은 null 리턴
    - 명령 프롬프트에서 반드시 실행
- Console 클래스의 읽기 메소드

리턴타입	메소드	설명
String	readLine()	엔터키를 입력하기 전의 모든 문자열을 읽음
char[]	readPassword()	키보드 입력 문자를 콘솔에 보여주지 않고 문자열을 읽음



ConsoleExample.java 아이디와 패스워드를 콘솔로부터 읽음

```
1  import java.io.Console;
2
3  public class ConsoleExample {
4      public static void main(String[] args) {
5          Console console = System.console();
6
7          System.out.print("아이디: ");
8          String id = console.readLine();
9
10         System.out.print("패스워드: ");
11         char[] charPass = console.readPassword();
12         String strPassword = new String(charPass);
13
14         System.out.println("----------------------");
15         System.out.println(id);
16         System.out.println(strPassword);
17     }
18 }
19
```



3절. 콘솔 입출력

❖ Scanner 클래스

▪ Console 클래스의 단점

- 문자열은 읽을 수 있지만 기본 타입(정수, 실수) 값을 바로 읽을 수 없음

▪ java.util.Scanner

- 콘솔로부터 기본 타입의 값을 바로 읽을 수 있음

```
Scanner scanner = new Scanner(System.in)
```

- 제공하는 메소드

리턴타입	메소드	설명
boolean	nextBoolean()	boolean(true/false) 값을 읽는다.
byte	nextByte()	byte 값을 읽는다.
short	nextShort()	short 값을 읽는다.
int	nextInt()	int 값을 읽는다.
long	nextLong()	long 값을 읽는다.
float	nextFloat()	float 값을 읽는다.
double	nextDouble()	double 값을 읽는다.
String	nextLine()	String 값을 읽는다.

ScannerExample.java 문자열, 정수, 실수를 직접 읽는 예제

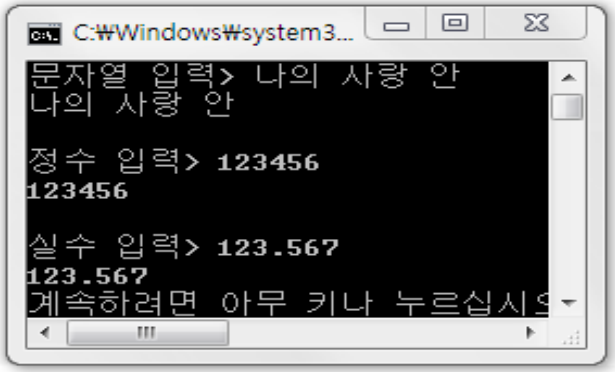
```java
import java.util.Scanner;

public class ScannerExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("문자열 입력> ");
        String inputString = scanner.nextLine();
        System.out.println(inputString);
        System.out.println();

        System.out.print("정수 입력> ");
        int inputInt = scanner.nextInt();
        System.out.println(inputInt);
        System.out.println();

        System.out.print("실수 입력> ");
        double inputDouble = scanner.nextDouble();
        System.out.println(inputDouble);
    }
}
```



4절. 파일 입출력

❖ File 클래스

- 파일 시스템의 파일을 표현하는 클래스
  - 파일 크기, 파일 속성, 파일 이름 등의 정보 제공
  - 파일 생성 및 삭제 기능 제공
  - 디렉토리 생성, 디렉토리에 존재하는 파일 리스트 얻어내는 기능 제공
- 파일 객체 생성

```
File file = new File("C:₩₩Temp₩₩file.txt");
File file = new File("C:/Temp/file.txt");
```

- 파일 또는 디렉토리 존재 유무 확인 메소드

```
boolean isExist = file.exists();
```

- 파일 및 디렉토리 생성 및 삭제 메소드

리턴타입	메소드	설명
boolean	createNewFile()	새로운 파일을 생성
booelan	mkdir()	새로운 디렉토리를 생성
boolean	mkdirs()	경로상에 없는 모든 디렉토리를 생성
boolean	delete()	파일 또는 디렉토리 삭제

4절. 파일 입출력

- **파일 및 디렉토리의 정보를 리턴하는 메소드**

리턴타입	메소드	설명
boolean	canExecute()	실행할 수 있는 파일인지 여부
boolean	canRead()	읽을 수 있는 파일인지 여부
boolean	canWrite()	수정 및 저장할 수 있는 파일인지 여부
String	getName()	파일의 이름을 리턴
String	getParent()	부모 디렉토리를 리턴
File	getParentFile()	부모 디렉토리를 File 객체로 생성후 리턴
String	getPath()	전체 경로를 리턴
boolean	isDirectory()	디렉토리인지 여부
boolean	isFile()	파일인지 여부
boolean	isHidden()	숨김 파일인지 여부
long	lastModified()	마지막 수정 날짜 및 시간을 리턴
long	length()	파일의 크기 리턴
String[]	list()	디렉토리에 포함된 파일 및 서브디렉토리 목록 전부를 String 배열로 리턴
String[]	list(FilenameFilter filter)	디렉토리에 포함된 파일 및 서브디렉토리 목록 중에 FilenameFilter 에 맞는 것만 String 배열로 리턴
File[]	listFiles()	디렉토리에 포함된 파일 및 서브 디렉토리 목록 전부를 File 배열로 리턴
File[]	listFiles(FilenameFilter filter)	디렉토리에 포함된 파일 및 서브디렉토리 목록 중에 FilenameFilter 에 맞는 것만 File 배열로 리턴

FileExample.java File 클래스를 이용한 파일 및 디렉토리 정보 출력

```java
import java.io.File;
import java.net.URI;
import java.text.SimpleDateFormat;
import java.util.Date;

public class FileExample {
    public static void main(String[] args) throws Exception {
        File dir = new File("C:/Temp/Dir");
        File file1 = new File("C:/Temp/file1.txt");
        File file2 = new File("C:/Temp/file2.txt");
        File file3 = new File(new URI("file:///C:/Temp/file3.txt"));

        if(dir.exists() == false) {  dir.mkdirs();  }
        if(file1.exists() == false) {  file1.createNewFile();  }
        if(file2.exists() == false) {  file2.createNewFile();  }
        if(file3.exists() == false) {  file3.createNewFile();  }

        File temp = new File("C:/Temp");
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd   a   HH:mm");
        File[] contents = temp.listFiles();
        System.out.println("날짜                  시간          형태          크기     이름");
        System.out.println("-------------------------------------------------------------------------");
        for(File file : contents) {
            System.out.print(sdf.format(new Date(file.lastModified())));
            if(file.isDirectory()) {
                System.out.print("\t<DIR>\t\t\t" + file.getName());
            } else {
                System.out.print("\t\t\t" + file.length() + "\t" + file.getName());
            }
            System.out.println();
        }
    }
}
```



4절. 파일 입출력

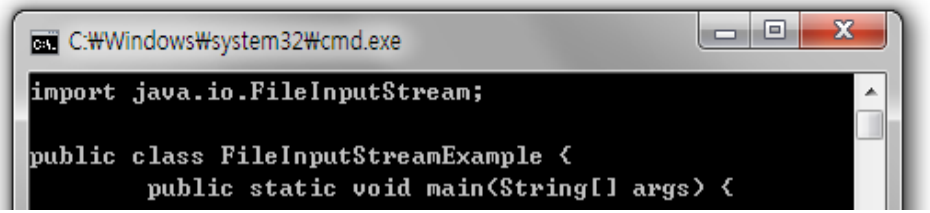
❖ FileInputStream

- 파일로부터 바이트 단위로 읽어 들일 때 사용
  - 그림, 오디오, 비디오, 텍스트 파일 등 모든 종류의 파일을 읽을 수 있음
- 객체 생성 방법
  - FileInputStream 객체가 생성될 때 파일과 직접 연결
  - 만약 파일이 존재하지 않으면 FileNotFoundException 발생
  - try-catch문으로 예외 처리
- InputStream 하위 클래스 - 사용 방법이 InputStream과 동일



FileInputExample.java 텍스트 파일을 읽고 출력

```
1  import java.io.FileInputStream;
2
3  public class FileInputStreamExample {
4      public static void main(String[] args) {
5          try {
6              FileInputStream fis = new FileInputStream("C:/Java/FileInputStreamExample.java");
7              int data;
8              while ( (data = fis.read() ) != -1 ) {
9                  System.out.write(data);
10             }
11             fis.close();
12         } catch(Exception e) {
13             e.printStackTrace();
14         }
15     }
16 }
17
18
```



4절. 파일 입출력

❖ FileOutputStream

- 파일에 바이트 단위로 데이터를 저장할 때 사용
  - 그림, 오디오, 비디오, 텍스트 등 모든 종류의 데이터를 파일로 저장

- 객체 생성 방법
  - 파일이 이미 존재할 경우, 데이터를 출력하게 되면 파일을 덮어쓰는 단점
  - 기존 파일 내용 끝에 데이터를 추가할 경우

```
FileOutputStream fis = new FileOutputStream("C:/Temp/data.txt", true);
FileOutputStream fis = new FileOutputStream(file, true);
```

- OutputStream 하위 클래스 – 사용 방법이 OutputStream과 동일



FileOutputStreamExample.java 파일 복사

```java
import java.io.FileInputStream;
import java.io.FileOutputStream;

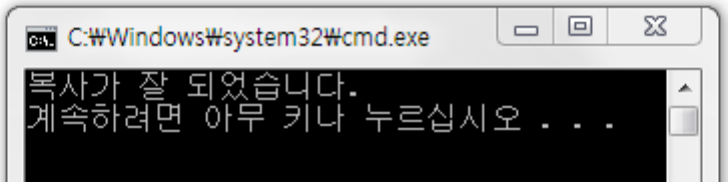
public class FileOutputStreamExample {
    public static void main(String[] args) throws Exception {
        String originalFileName = "C:/java/house.jpg";
        String targetFileName = "C:/Temp/house.jpg";

        FileInputStream fis = new FileInputStream(originalFileName);
        FileOutputStream fos = new FileOutputStream(targetFileName);

        int readByteNo;
        byte[] readBytes = new byte[100];
        while( (readByteNo = fis.read(readBytes)) != -1 ) {
            fos.write(readBytes, 0, readByteNo);
        }

        fos.flush();
        fos.close();
        fis.close();

        System.out.println("복사가 잘 되었습니다.");
    }
}
```



4절. 파일 입출력

❖ FileReader

- 텍스트 파일로부터 데이터를 읽어 들일 때 사용
  - 문자 단위로 읽음
    - 텍스트가 아닌 그림, 오디오, 비디오 등의 파일은 읽을 수 없음
- 객체 생성 방법

```
//방법 1
FileReader fr = new FileReader("C:/Temp/file.txt");

//방법 2
File file = new File("C:/Temp/file.txt");
FileReader fr = new FileReader(file);
```

  - FileReader 객체가 생성될 때 파일과 직접 연결
  - 만약 파일이 존재하지 않으면 FileNotFoundException 발생
  - try-catch문으로 예외 처리
- Reader 하위 클래스 - 사용 방법 Reader와 동일

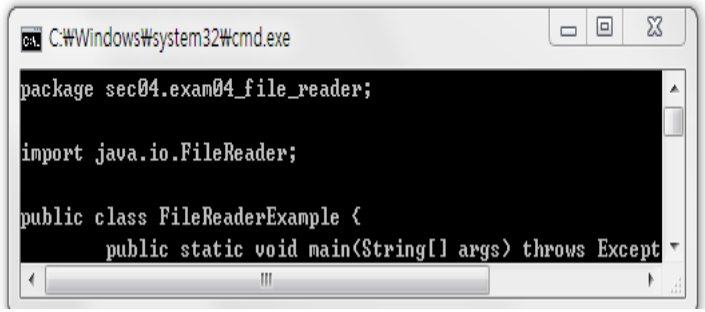


FileReaderExample.java 테스트 파일 읽기

```java
import java.io.FileReader;

public class FileReaderExample {
    public static void main(String[] args) throws Exception {
        FileReader fr = new FileReader("C:/JavaProgramming/source/chap18/src/sec04/exam04_file_reader/FileReaderExample.java");

        int readCharNo;
        char[] cbuf = new char[100];
        while ((readCharNo=fr.read(cbuf)) != -1) {
            String data = new String(cbuf, 0, readCharNo);
            System.out.print(data);
        }
        fr.close();
    }
}
```



4절. 파일 입출력

❖ FileWriter

- 텍스트 파일에 문자 데이터를 저장할 때 사용
  - 텍스트가 아닌 그림, 오디오, 비디오 등의 데이터를 파일로 저장 불가
- 객체 생성 방법
  - 파일이 이미 존재할 경우, 데이터를 출력하게 되면 파일을 덮어쓰게 됨.
    - 파일 존재여부 따라 분기
  - 기존 파일 내용 끝에 데이터를 추가할 경우

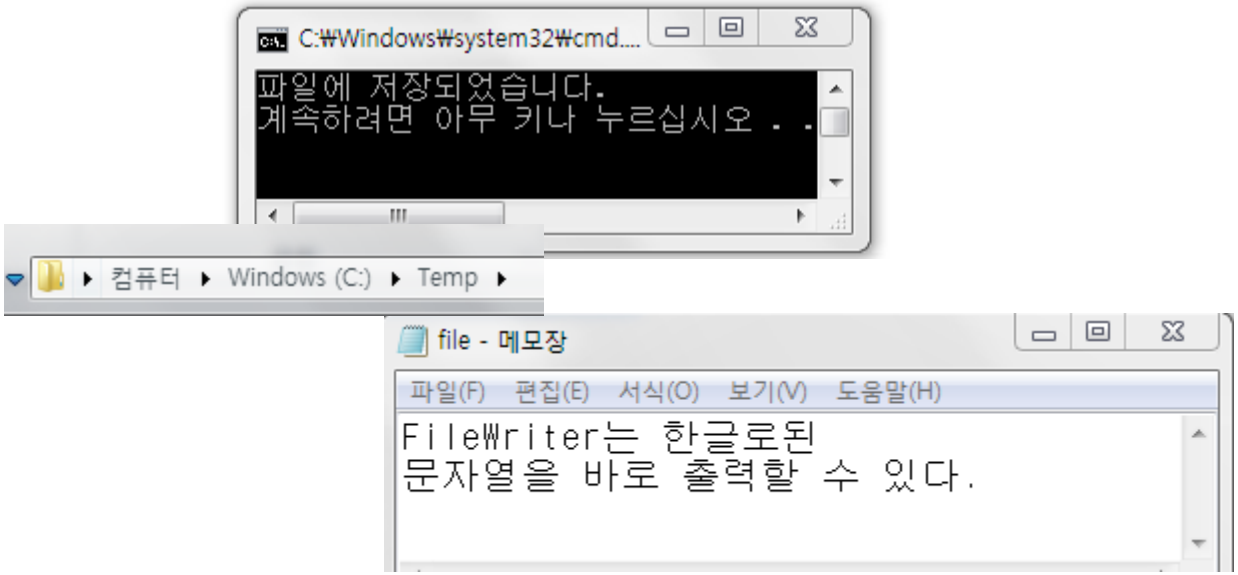
```
FileWriter fw = new FileWriter("C:/Temp/file.txt", true);
FileWriter fw = new FileWriter(file, true);
```

- Writer 하위 클래스 – 사용 방법이 Writer와 동일



FileWriterExample.java 문자열을 파일에 저장

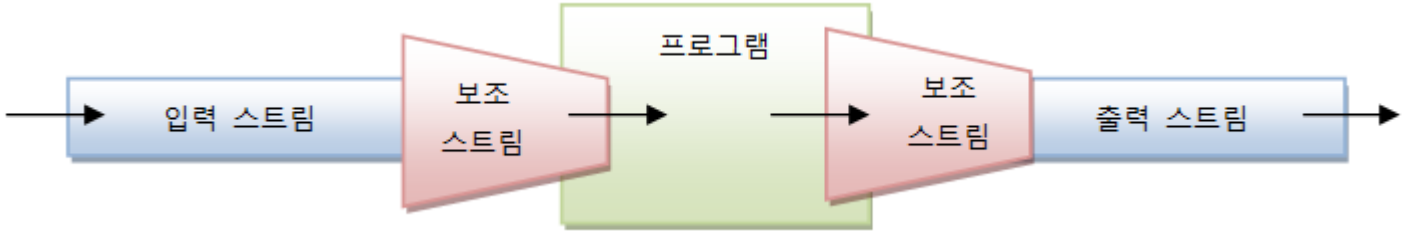
```
1  import java.io.File;
2  import java.io.FileWriter;
3
4  public class FileWriterExample {
5      public static void main(String[] args) throws Exception {
6          File file = new File("C:/Temp/file.txt");
7          FileWriter fw = new FileWriter(file, true);
8          fw.write("FileWriter는 한글로된 " + "\r\n");
9          fw.write("문자열을 바로 출력할 수 있다." + "\r\n");
10         fw.flush();
11         fw.close();
12         System.out.println("파일에 저장되었습니다.");
13     }
14 }
15
16
```



5절. 보조 스트림

❖ 보조 스트림

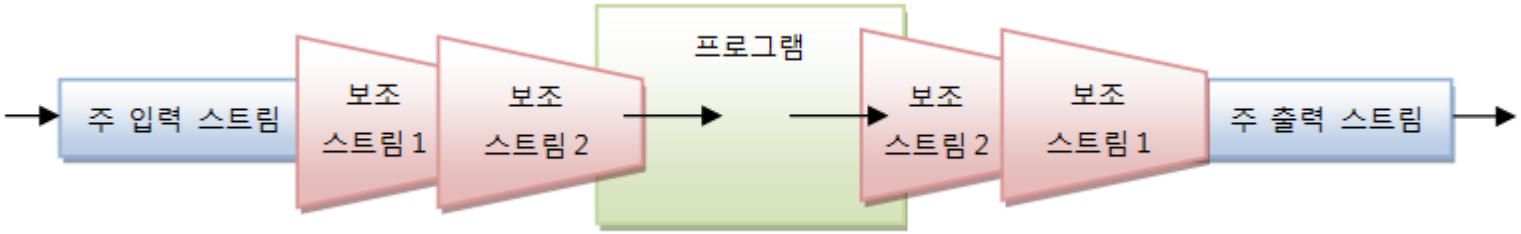
- 다른 스트림과 연결 되어 여러 가지 편리한 기능을 제공해주는 스트림
  - 문자 변환, 입출력 성능 향상, 기본 데이터 타입 입출력, 객체 입출력 등의 기능을 제공



- 보조 스트림 생성

```
보조스트림 변수 = new 보조스트림(연결스트림)
```

- 보조 스트림 체인 – 다른 보조 스트림과 연결되어 역할 수행

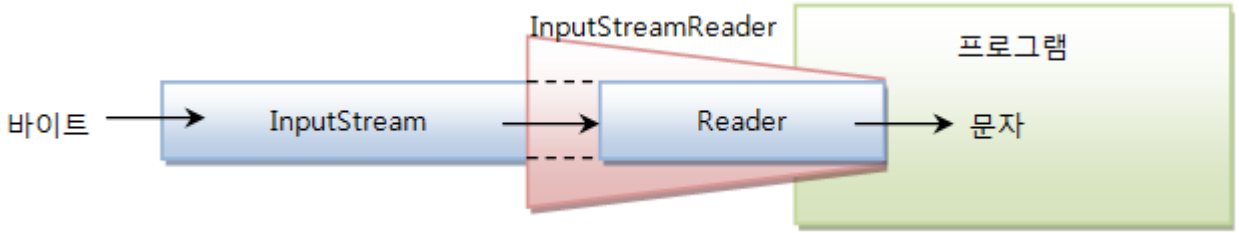


5절. 보조 스트림

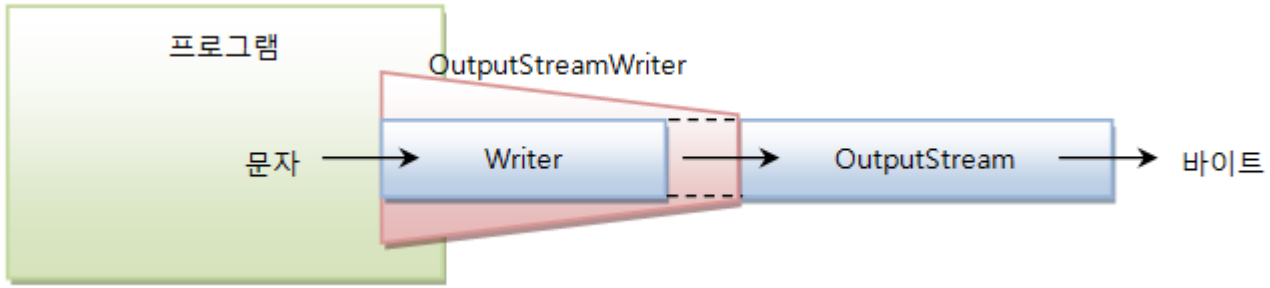
❖ 문자 변환 보조 스트림

- 소스 스트림이 바이트 기반 스트림이지만 데이터가 문자일 경우 사용
  - Reader와 Writer는 문자 단위로 입출력 – 바이트 기반 스트림보다 편리
  - 문자셋의 종류를 지정할 수 있기 때문에 다양한 문자 입출력 가능

- InputStreamReader

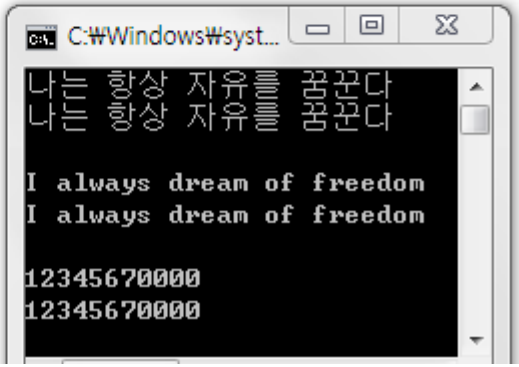


- OutputStreamWriter



InputStreamReaderExample.java 콘솔에서 한글 입력받기

```
1  import java.io.InputStream;
2  import java.io.InputStreamReader;
3  import java.io.Reader;
4
5  public class InputStreamReaderExample {
6      public static void main(String[] args) throws Exception {
7          InputStream is = System.in;
8          Reader reader = new InputStreamReader(is);
9
10         int readCharNo;
11         char[] cbuf = new char[100];
12         while ((readCharNo=reader.read(cbuf)) != -1) {
13             String data = new String(cbuf, 0, readCharNo);
14             System.out.println(data);
15         }
16
17         reader.close();
18     }
19 }
20
21
```



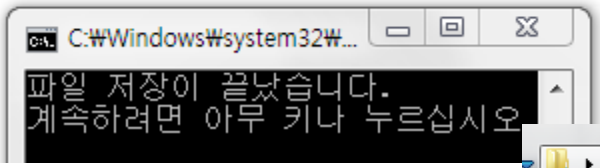
OutputStreamWriterExample.java 파일로 출력하기

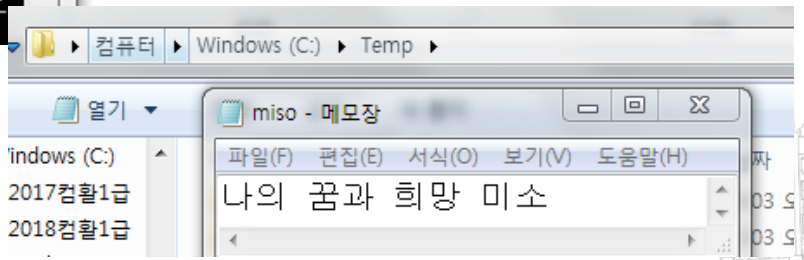
```java
import java.io.FileOutputStream;
import java.io.InputStreamReader;
import java.io.OutputStreamWriter;
import java.io.Reader;
import java.io.Writer;

public class OutputStreamWriterExample {
    public static void main(String[] args) throws Exception {
        FileOutputStream fos = new FileOutputStream("C:/Temp/miso.txt");
        Writer writer = new OutputStreamWriter(fos);

        String data = "나의 꿈과 희망 미소";
        writer.write(data);

        writer.flush();
        writer.close();
        System.out.println("파일 저장이 끝났습니다.");
    }
}
```



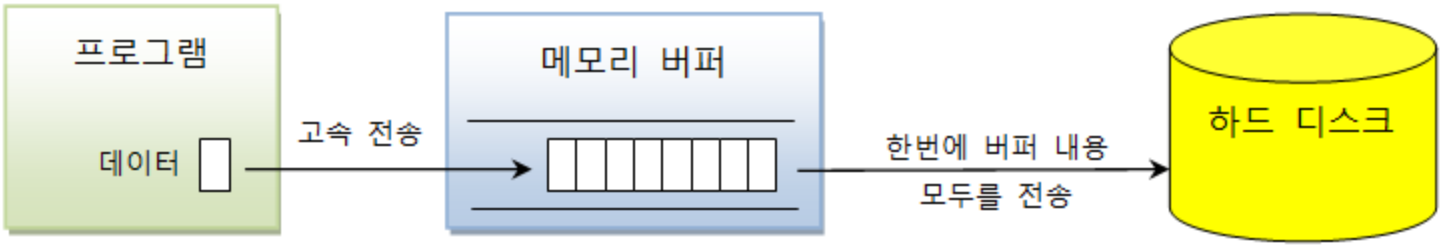


5절. 보조 스트림

❖ 성능 향상 보조 스트림

- 입출력 성능에 영향을 미치는 입출력 소스
  - 하드 디스크
  - 느린 네트워크

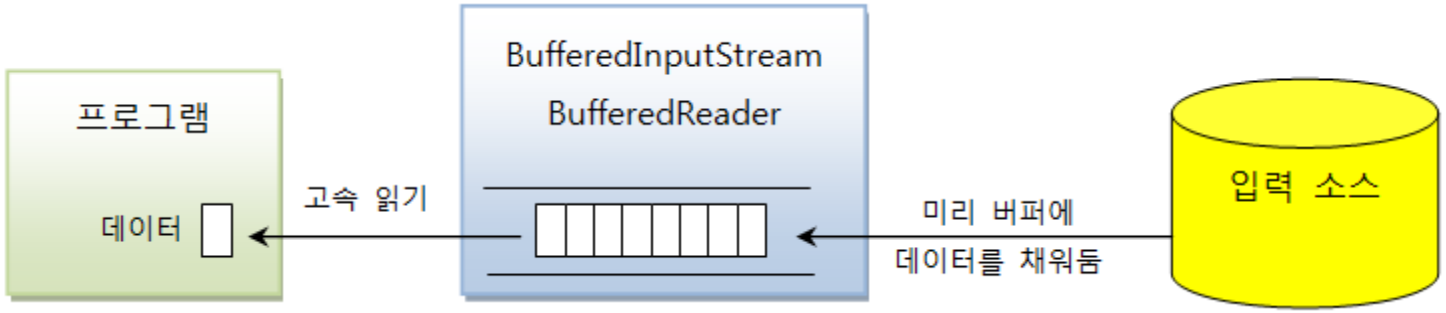
- 버퍼를 이용한 해결 (p.1032~1037)
  - 입출력 소스와 직접 작업하지 않고 버퍼(buffer)와 작업 - 실행 성능 향상



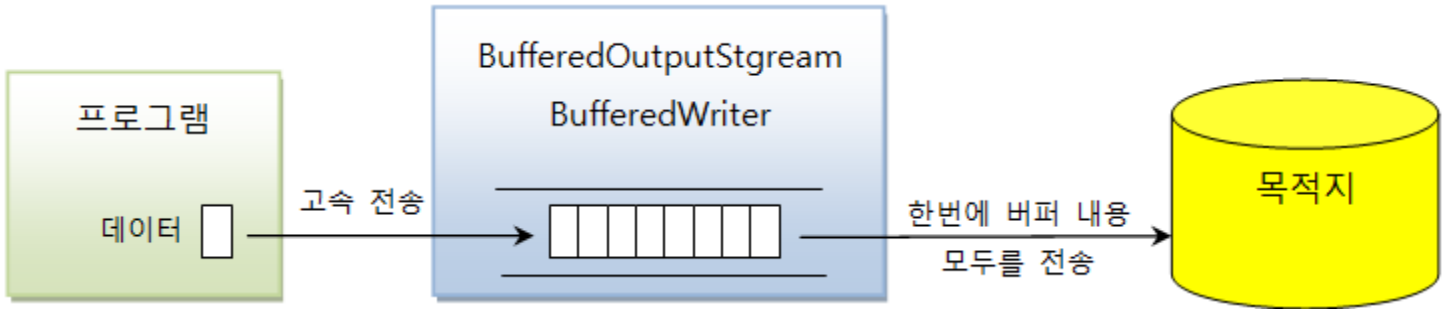
  - 프로그램은 쓰기 속도 향상
  - 버퍼 차게 되면 데이터를 한꺼번에 하드 디스크로 보내 출력 횟수를 줄여줌

5절. 보조 스트림

- **BufferedInputStream, BufferedReader**



- **BufferedOutputStream과 BufferedWriter**



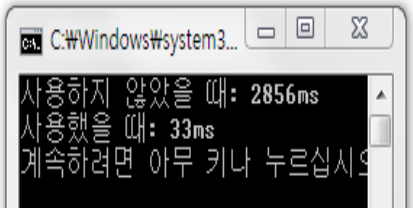
BufferedInputStreamExample.java 버퍼를 사용했을 때의 성능 테스트

```java
import java.io.BufferedInputStream;
import java.io.FileInputStream;

public class BufferedInputStreamExample {
    public static void main(String[] args) throws Exception {
        long start = 0;
        long end = 0;

        FileInputStream fis1 = new FileInputStream("C:/JavaProgramming/source/chap18/src/sec05/exam03_bufferedinputstream/forest.jpg");
        start = System.currentTimeMillis();
        while(fis1.read() != -1) {}
        end = System.currentTimeMillis();
        System.out.println("사용하지 않았을 때: " + (end-start) + "ms");
        fis1.close();

        FileInputStream fis2 = new FileInputStream("C:/JavaProgramming/source/chap18/src/sec05/exam03_bufferedinputstream/forest.jpg");
        BufferedInputStream bis = new BufferedInputStream(fis2);
        start = System.currentTimeMillis();
        while(bis.read() != -1) {}
        end = System.currentTimeMillis();
        System.out.println("사용했을 때: " + (end-start) + "ms");
        bis.close();
        fis2.close();
    }
}
```



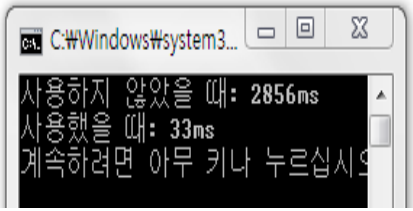
BufferedOutputStreamExample.java 버퍼를 사용했을 때의 성능 테스트

```java
import java.io.BufferedInputStream;
import java.io.FileInputStream;

public class BufferedInputStreamExample {
    public static void main(String[] args) throws Exception {
        long start = 0;
        long end = 0;

        FileInputStream fis1 = new FileInputStream("C:/JavaProgramming/source/chap18/src/sec05/exam03_bufferedinputstream/forest.jpg");
        start = System.currentTimeMillis();
        while(fis1.read() != -1) {}
        end = System.currentTimeMillis();
        System.out.println("사용하지 않았을 때: " + (end-start) + "ms");
        fis1.close();

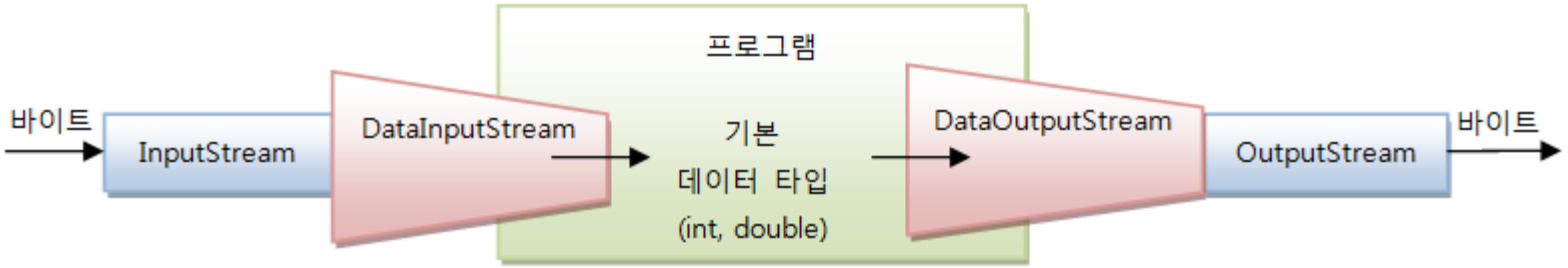
        FileInputStream fis2 = new FileInputStream("C:/JavaProgramming/source/chap18/src/sec05/exam03_bufferedinputstream/forest.jpg");
        BufferedInputStream bis = new BufferedInputStream(fis2);
        start = System.currentTimeMillis();
        while(bis.read() != -1) {}
        end = System.currentTimeMillis();
        System.out.println("사용했을 때: " + (end-start) + "ms");
        bis.close();
        fis2.close();
    }
}
```



5절. 보조 스트림

❖ 기본 타입 입출력 보조 스트림

- 입출력 순서를 맞추어 사용



DataInputStream		DataOutputStream	
boolean	readBoolean()	void	writeBoolean(boolean v)
byte	readByte()	void	writeByte(int v)
char	readChar()	void	writeChar(int v)
double	readDouble()	void	writeDouble(double v)
float	readFloat()	void	writeFloat(float v)
int	readInt()	void	writeInt(int v)
long	readLong()	void	writeLong(long v)
short	readShort()	void	writeShort(int v)
String	readUTF()	void	writeUTF(String str)

DataInputOutputStreamExample.java 기본 데이터 타입 입출력

```java
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.FileInputStream;
import java.io.FileOutputStream;

public class DataInputOutputStreamExample {
    public static void main(String[] args) throws Exception {
        FileOutputStream fos = new FileOutputStream("C:/Temp/primitive.dat");
        DataOutputStream dos = new DataOutputStream(fos);

        dos.writeUTF("홍길동");
        dos.writeDouble(95.5);
        dos.writeInt(1);

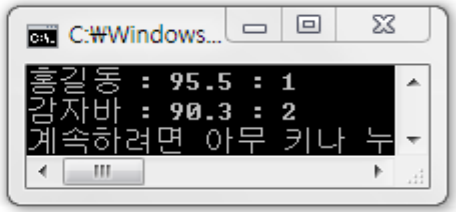
        dos.writeUTF("감자바");
        dos.writeDouble(90.3);
        dos.writeInt(2);

        dos.flush(); dos.close(); fos.close();

        FileInputStream fis = new FileInputStream("C:/Temp/primitive.dat");
        DataInputStream dis = new DataInputStream(fis);

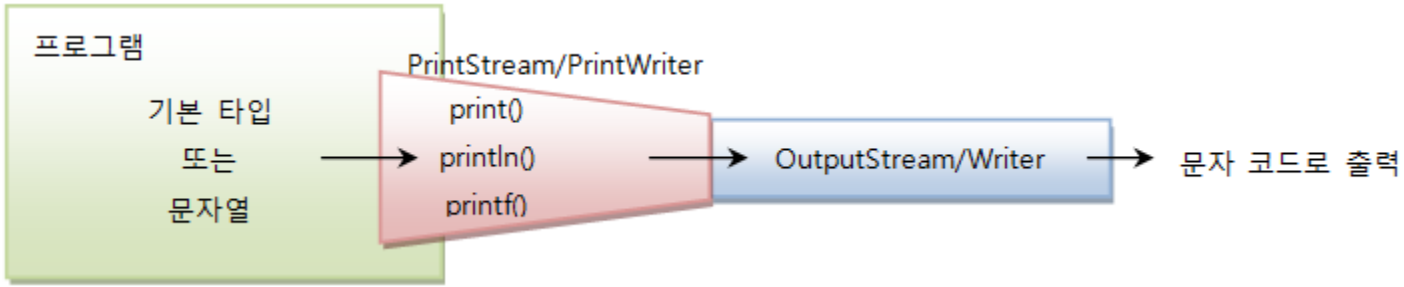
        for(int i=0; i<2; i++) {
            String name = dis.readUTF();
            double score = dis.readDouble();
            int order = dis.readInt();
            System.out.println(name + " : " + score + " : " + order);
        }

        dis.close(); fis.close();
    }
}
```



5절. 보조 스트림

❖ 프린터 보조 스트림

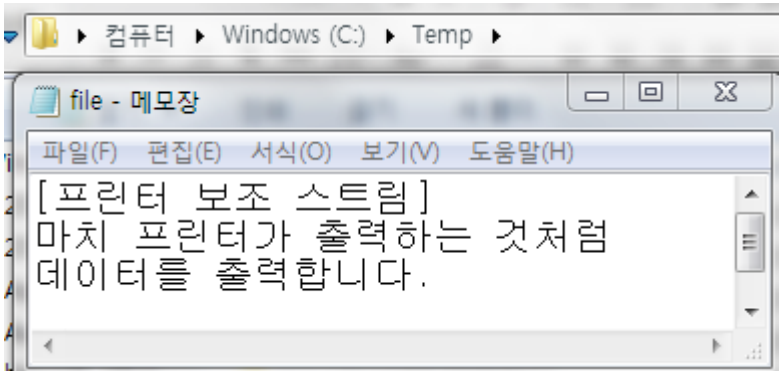


PrintStream / PrintWriter			
void	print(boolean b)	void	println(boolean b)
void	print(char c)	void	println(char c)
void	print(double d)	void	println(double d)
void	print(float f)	void	println(float f)
void	print(int i)	void	println(int i)
void	print(long l)	void	println(long l)
void	print(Object obj)	void	println(Object obj)
void	print(String s)	void	println(String s)
		void	println()

- **println()은 데이터 끝에 개행문자 추가, printf는 format string 출력**

PrintStreamExample.java 라인 단위로 출력하기

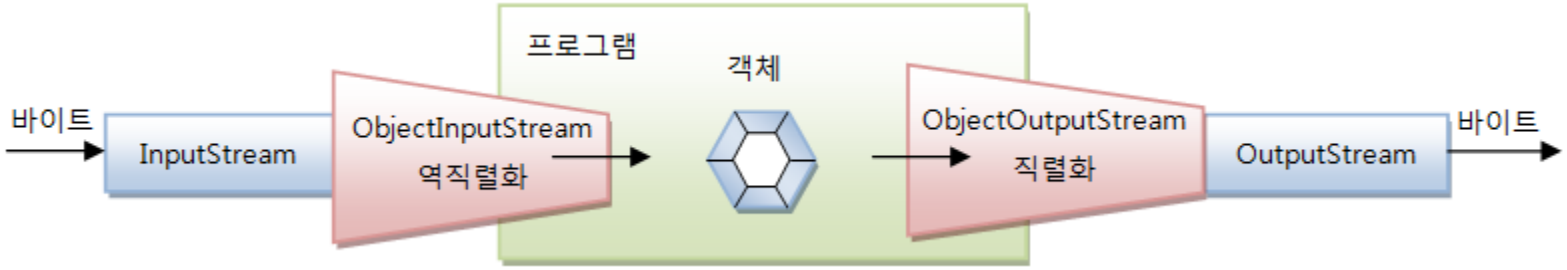
```
1  import java.io.FileOutputStream;
2  import java.io.PrintStream;
3
4  public class PrintStreamExample {
5      public static void main(String[] args) throws Exception {
6          FileOutputStream fos = new FileOutputStream("C:/Temp/file.txt");
7          PrintStream ps = new PrintStream(fos);
8
9          ps.println("[프린터 보조 스트림]");
10         ps.print("마치 ");
11         ps.println("프린터가 출력하는 것처럼 ");
12         ps.println("데이터를 출력합니다.");
13
14         ps.flush();
15         ps.close();
16         fos.close();
17     }
18 }
```



5절. 보조 스트림

❖ 객체 입출력 보조 스트림

- 객체를 파일 또는 네트워크로 입출력할 수 있는 기능 제공
- 객체 직렬화
  - 객체는 문자가 아니므로 바이트 기반 스트림으로 데이터 변경 필요
- ObjectInputStream, ObjectOutputStream



- 직렬화가 가능한 클래스(Serializable)
  - 자바에서는 Serializable 인터페이스를 구현한 클래스만 직렬화 할 수 있도록 제한, transient 필드는 제외
  - 객체 직렬화 할 때 private 필드 포함한 모든 필드를 바이트로 변환 가능

ObjectInputOutputStreamExample.java 다양한 객체를 쓰고 읽기

```java
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;

public class ObjectInputOutputStreamExample {
    public static void main(String[] args) throws Exception {
        FileOutputStream fos = new FileOutputStream("C:/Temp/Object.dat");
        ObjectOutputStream oos = new ObjectOutputStream(fos);

        oos.writeObject(new Integer(10));
        oos.writeObject(new Double(3.14));
        oos.writeObject(new int[] { 1, 2, 3 });
        oos.writeObject(new String("홍길동"));

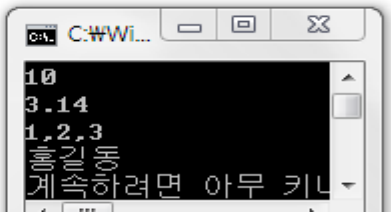
        oos.flush();    oos.close(); fos.close();

        FileInputStream fis = new FileInputStream("C:/Temp/Object.dat");
        ObjectInputStream ois = new ObjectInputStream(fis);

        Integer obj1 = (Integer) ois.readObject();
        Double obj2 = (Double) ois.readObject();
        int[] obj3 = (int[]) ois.readObject();
        String obj4 = (String) ois.readObject();

        ois.close(); fis.close();

        System.out.println(obj1);
        System.out.println(obj2);
        System.out.println(obj3[0] + "," + obj3[1] + "," + obj3[2]);
        System.out.println(obj4);
    }
}
```



5절. 보조 스트림

- serialVersionUID 필드 (p.1047~1049)
  - 직렬화된 객체를 역직렬화 할 때는 직렬화 했을 때와 같은 클래스 사용
  - 클래스의 이름이 같더라도 클래스의 내용이 변경된 경우 역직렬화 실패
  - serialVersionUID
    - 같은 클래스임을 알려주는 식별자 역할
    - Serializable 인터페이스 구현
      - » 컴파일 시 자동적으로 serialVersionUID 정적 필드 추가
    - 재컴파일하면 serialVersionUID의 값 변경
  - 불가피한 수정 있을 경우 명시적으로 serialVersionUID 선언

```
static final long serialVersionUID = 정수값;
```



ClassA.java 직렬화가 가능한 클래스

```java
import java.io.Serializable;

public class ClassA implements Serializable {
    int field1;
    ClassB field2 = new ClassB();
    static int field3;
    transient int field4;
}
```

ClassB.java 직렬화가 가능한 클래스

```java
import java.io.Serializable;

public class ClassB implements Serializable {
    int field1;
}
```



SerializableWriter.java 직렬화에서 파일에 저장

```
1  import java.io.FileInputStream;
2  import java.io.FileOutputStream;
3  import java.io.ObjectInputStream;
4  import java.io.ObjectOutputStream;
5
6  public class SerializableWriter {
7      public static void main(String[] args) throws Exception {
8          FileOutputStream fos = new FileOutputStream("C:/Temp/Object.dat");
9          ObjectOutputStream oos = new ObjectOutputStream(fos);
10         ClassA classA = new ClassA();
11         classA.field1 = 1;
12         classA.field2.field1 = 2;
13         classA.field3 = 3;
14         classA.field4 = 4;
15         oos.writeObject(classA);
16         oos.flush();    oos.close(); fos.close();
17     }
18 }
```



SerializableReader.java 역질력화해서 복원된 필드 조사

```java
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;

public class SerializableReader {
    public static void main(String[] args) throws Exception {
        FileInputStream fis = new FileInputStream("C:/Temp/Object.dat");
        ObjectInputStream ois = new ObjectInputStream(fis);
        ClassA v = (ClassA) ois.readObject();
        System.out.println("field1: " + v.field1);
        System.out.println("field2.field1: " + v.field2.field1);
        System.out.println("field3: " + v.field3);
        System.out.println("field4: " + v.field4);
    }
}

```

```
field1: 1
field2.field1: 2
field3: 0
field4: 0
계속하려면 아무 키
```

5절. 보조 스트림

- writeObject()와 readObject() 메소드
  - writeObject(ObjectOutputStream out)
    - 직렬화 직전 자동 호출
    - 추가 직렬화할 내용 작성 가능
  - readObject(ObjectInputStream in)
    - 역직렬화 직전 자동 호출
    - 추가 역직렬화 내용 작성 가능
  - 추가 직렬화 및 역직렬화 필요한 경우
    - 부모 클래스가 Serializable 구현하지 않고, 자식 클래스가 Serializable 구현한 경우
    - 부모 필드는 직렬화에서 제외
      - writeObject() 에서 부모 필드 직렬화 필요
      - readObject()에서 부모 필드 역직렬화 필요
      - 부모 클래스가 Serializable 구현하도록 하는 게 제일 쉬움

Parent.java Serializable을 구현하지 않은 부모 클래스

```java
public class Parent {
    public String field1;
}

```

Child.java 직렬화되지 않는 부모 클래스의 필드 처리

```java
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;

public class Child extends Parent implements Serializable {
    public String field2;

    private void writeObject(ObjectOutputStream out) throws IOException {
        out.writeUTF(field1);
        out.defaultWriteObject();
    }

    private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException {
        field1 = in.readUTF();
        in.defaultReadObject();
    }
}
```

NonSerializableParentExample.java 직렬화 및 역직렬화

```java
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;

public class NonSerializableParentExample {
    public static void main(String[] args) throws Exception {
        FileOutputStream fos = new FileOutputStream("C:/Temp/Object.dat");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        Child child = new Child();
        child.field1 = "홍길동";
        child.field2 = "홍삼원";
        oos.writeObject(child);
        oos.flush(); oos.close(); fos.close();

        FileInputStream fis = new FileInputStream("C:/Temp/Object.dat");
        ObjectInputStream ois = new ObjectInputStream(fis);
        Child v = (Child) ois.readObject();
        System.out.println("field1: " + v.field1);
        System.out.println("field2: " + v.field2);
        ois.close(); fis.close();
    }
}
```

